```
key = newKey;     #B
```

#A Overwrite the old key with zero bytes
#B Replace the old key with the new key

> **TIP** In Java and similar languages, the garbage collector may duplicate the contents of variables in memory, so copies may remain even if you attempt to wipe the data. You can use `ByteBuffer.allocateDirect()` to create *off-heap memory* that is not managed by the garbage collector.

Ratcheting only works if both the client and the server can determine when a ratchet occurs, otherwise they will end up using different keys. You should therefore perform ratchet operations at well-defined moments, for example each device might ratchet its key at midnight every day, or every hour, or perhaps even after every 10 messages.[11] The rate at which ratchets should be performed depends on the number of requests that the device sends, and the sensitivity of the data being transmitted.

Ratcheting after a fixed number of messages can help to detect compromise: if an attacker is using a device's stolen secret key, then the API server will receive extra messages in addition to any the device sent and so will perform the ratchet earlier than the legitimate device. If the device discovers that the server is performing ratcheting earlier than expected, then this is evidence that another party has compromised the device secret key.